

GOLEM Assignment 3

Final Submission Report

Akash Gupta (23110020)
Kaushal Bule (23110160)
Siddhesh Umarjee (23110347)

1 Introduction and Scope

This report presents our complete work for Assignment 3 of the GOLEM (Project Repository) compiler project. The goal of this assignment was not just to build a parser, but to understand its behavior deeply — from identifying ambiguities to refining the grammar and improving usability.

We systematically cover:

- Construction and analysis of an LALR(1) parser
- Identification and resolution of parsing conflicts
- Tracking derivations during parsing
- Extracting and visualizing parsing tables
- Designing robust error diagnostics

The report reflects the current state of our implementation and all generated artifacts.

2 Language Overview

GOLEM (Grid-Oriented Logic for Entity Manipulation) is a domain-specific language designed for specifying the behavior of agents operating in a 2D grid.

2.1 Core Concepts

At a high level, the language allows us to:

- Define grids with dimensions
- Place obstacles at specific coordinates
- Define reusable behaviors using `blueprint`
- Initialize execution via `construct` and `spawn`

2.2 Supported Statements

The language supports a variety of action and control constructs:

- Movement: `go expr;`
- Rotation: `turn direction;`
- Conditional execution using `if/else`
- Repetition using `repeat expr stmt`
- Interaction actions such as `pick` and `drop`

2.3 Expression Support

Arithmetic expressions are supported with standard precedence:

`+, -, ×, ÷, %`

along with unary minus. These expressions are used across dimensions, movement counts, and control flow.

3 Toolchain and Build Setup

3.1 Required Tools

- `bison`, `flex`, `gcc`
- `make`, `python3`
- `graphviz` (for automaton visualization)

3.2 Build Commands

From the repository root:

```
make
make test
make clean
```

Question-specific builds:

```
make Q1
make Q2
```

4 Q1: LALR(1) Automaton with Conflicts

4.1 Objective

The goal here was to construct the LALR(1) automaton from an intentionally ambiguous grammar and carefully examine the resulting conflicts.

4.2 Approach

We used Bison's verbose mode to generate detailed state and conflict information, and complemented this with Graphviz visualizations for better intuition.

4.3 Results

- Total states: 132
- Total conflicts: 10
- State 102: 1 shift/reduce conflict
- State 107: 9 reduce/reduce conflicts

4.4 Build

```
cd Q1_LALR_With_Conflicts
bison -v -d golem_with_conflicts.y
flex golem.l
gcc -Wall -g -o golem_with_conflicts lex.yy.c y.tab.c -ll
```

4.5 Discussion

The conflicts observed were expected:

- The shift/reduce conflict corresponds to the classic dangling **else**.
- The reduce/reduce conflicts arise from ambiguity in block structure.

This stage served as a baseline for understanding how ambiguity manifests in parser construction.

5 Q2: Conflict Resolution

5.1 Objective

The aim was to eliminate all conflicts while preserving the intended semantics of the language.

5.2 Strategy

Dangling Else We resolved ambiguity using precedence:

- `%nonassoc THEN`
- `%nonassoc ELSE`
- `%prec THEN`

Block Structure We refactored the grammar to clearly distinguish compound statements, avoiding overlapping reductions.

5.3 Results

- Conflicts reduced: $10 \rightarrow 0$
- States reduced: $132 \rightarrow 128$

5.4 Build

```
cd Q2_Conflict_Resolution
bison -v -d golem.y
flex golem.l
gcc -Wall -g -o golem golem.tab.c lex.yy.c -lfl
```

5.5 Discussion

After resolution:

- `else` correctly binds to the nearest unmatched `if`
- Nested control structures behave as expected

6 Q3: Reverse Derivation Tracking

6.1 Objective

To make the parsing process more transparent, we recorded reductions and displayed them in reverse order.

6.2 Implementation

We used a linked-list structure where each node stores:

- Production rule description
- Rule number

Core functions:

- `add_derivation()`
- `display_reverse_derivation()`

6.3 Results

- 57 productions tracked
- Clear bottom-up derivation sequence generated

6.4 Insight

This feature proved extremely useful for debugging and understanding how the parser constructs the final parse.

7 Q4: Parsing Table Extraction

7.1 Objective

Convert Bison output into structured parsing tables.

7.2 Approach

We implemented a Python script to extract:

- ACTION table
- GOTO table

7.3 Results

- States: 128
- ACTION columns: 47
- GOTO columns: 23

7.4 Command

```
python3 parsing_table.py <golem.output> <output_dir>
```

7.5 Interpretation

- sN : shift to state N
- rN : reduce by rule N
- acc: accept

8 Q5: Error Diagnostics

8.1 Objective

Enhance the parser to provide meaningful and recoverable error messages.

8.2 Key Features

- Line tracking using `%option yylineno`
- Verbose error messages
- Categorized diagnostics

8.3 Error Types

- Syntax errors (e.g., missing delimiters)
- Semantic errors (e.g., division by zero)
- Warnings (e.g., zero-iteration loops)

8.4 Build

```
cd Q5_Error_Diagnostics
bison -d golem.y
flex golem.l
gcc -Wall -g -o golem golem.tab.c lex.yy.c -lfl
./golem < test_errors.golem
```

8.5 Discussion

Instead of stopping at the first error, the parser continues and reports multiple issues, significantly improving usability.

9 Q6: Consolidated Notes

9.1 Key Additions

- Full arithmetic expression support
- Explicit handling of `if-else` ambiguity
- Unified execution workflow

9.2 Summary Table

Part	States	Conflicts	Focus
Q1	132	10	Ambiguity detection
Q2	128	0	Conflict resolution
Q3	–	–	Derivation tracking
Q4	128	0	Parsing tables
Q5	–	–	Diagnostics

10 Conclusion

This assignment walks through the full lifecycle of parser development:

1. Starting with an ambiguous grammar
2. Identifying and resolving conflicts
3. Understanding parser behavior via derivations
4. Making internal structures visible through tables
5. Improving usability with diagnostics

Overall, the final parser is:

- Conflict-free
- Fully functional
- Equipped with strong debugging and reporting tools

11 Reproducibility

```
make clean && make
make test

make Q1
make Q2

python3 Q4_Parsing_Table/parsing_table.py golem.output Q4_Parsing_Table
/
```

```
cd Q5_Error_Diagnostics  
./golem < test_errors.golem
```